

Class 10: Python — Exploratory Data Analysis

Data-driven Systems Engineering

Agenda

- Why Python is the working language of this course.
- From business question to exploratory workflow.
- Data quality dimensions and common failure modes.
- EDA patterns for tabular, imbalanced, and streaming-flavored data.
- How notebooks, pandas, and plotting libraries fit the engineering pipeline.

Learning goals

- Explain why Python is effective for analysis, prototyping, and deployment.
- Organize exploratory work into a reproducible ML workflow.
- Evaluate data quality before talking about model quality.
- Produce an EDA output that supports later feature engineering and modeling.

Course Map and Running Cases



Today in the course

Focus: Data understanding and exploratory analysis.

Bridge: This class moves from the business question into the data layer and prepares the observations we will later encode as features.

Fraud case

In the fraud case, EDA reveals imbalance, unusual value ranges, and the kinds of summaries needed before trustworthy modeling.

Pasteurization case

In the pasteurization case, EDA means inspecting sensor ranges, process states, and temporal patterns before building alerts or predictors.

Course thread: The output of this class is not a model yet; it is a credible understanding of what the data can and cannot support.

Why Python remains central

- One language can cover data ingestion, analysis, modeling, APIs, dashboards, and automation.
- The ecosystem is broad enough for both teaching and production-oriented prototypes.
- The syntax is readable, which matters when notebooks become shared learning artifacts.
- Python also integrates well with external services, files, and web frameworks.

Important nuance

Python is not always the fastest runtime, but it is often the fastest route from idea to tested workflow.

From problem statement to notebook workflow

1. Define the decision or question.
2. Identify the dataset and its scope.
3. Check data quality and missingness.
4. Summarize distributions and relationships.
5. Form hypotheses for feature engineering and modeling.
6. Record risks, assumptions, and next steps.

Repo activity

'notebooks/Class2_EDA.ipynb' and 'Class2_EDA_Exercise.ipynb' already illustrate this transition from dataset inspection to structured analysis.

Notebook step 1: load the Iris dataset

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")
iris.head()
```

- The notebook begins with a compact import-and-load step so students can start exploration immediately.
- Iris is intentionally simple: four numeric features and one class label.
- This makes it easier to focus on process before complexity.

Notebook step 2: inspect structure and missingness

```
iris.info()  
iris.isnull().sum()
```

- 'info()' summarizes schema, data types, and non-null counts.
- 'isnull().sum()' turns missingness into a concrete per-column check.
- Even when the dataset is clean, this step trains a habit students will need later.

KDD and CRISP-DM in practical terms

KDD emphasis

- Knowledge discovery
- Data-driven pattern finding
- Strong focus on analysis steps

CRISP-DM emphasis

- Business understanding first
- Structured iterative workflow
- Better project communication

Course perspective

Students should know both, but CRISP-DM is usually easier to communicate in team projects because it begins from a decision need rather than a dataset alone.

Data quality is an engineering topic

Dimension	What to verify
Accuracy	Are values plausible in the real domain?
Completeness	Are essential fields missing, null, or defaulted?
Consistency	Do categories, units, and timestamps match across sources?
Timeliness	Is the data still relevant for the decision we support?
Bias	Which users, behaviors, or operating states are underrepresented?

Better examples for EDA discussion

- Fraud detection: class imbalance means plain accuracy can be dangerously misleading.
- Titanic: missing values and categorical variables force explicit preprocessing choices.
- Wine or Iris: useful for technique learning, but too clean to represent production reality.
- Sensor streams: the same measurement may be normal in one operating state and abnormal in another.

What pandas contributes to the workflow

- Loading and inspecting structured data quickly.
- Profiling missing values, duplicates, and suspicious categories.
- Reshaping, grouping, joining, and aggregating observations.
- Time-aware analysis for event or sensor datasets.

Why it matters

If students cannot express data transformations clearly in pandas, they will struggle to make preprocessing reproducible later.

What plotting contributes

Matplotlib

Fine-grained control for reliable static plots and diagnostics.

seaborn

Faster statistical plots for distributions, pairwise relations, and summaries.

Notebook output

Good for exploratory storytelling, but should still be curated.

- Use plots to test hypotheses, not to decorate the notebook.
- Every plot should answer a question: imbalance, drift hint, correlation, or anomaly.

Notebook step 3: visualize distributions and relations

```
sns.pairplot(iris, hue="species", diag_kind="kde")  
plt.show()  
  
fig, axes = plt.subplots(2, 2, figsize=(12, 8))  
sns.boxplot(x="species", y="sepal_length", data=iris, ax=axes[0, 0])  
sns.boxplot(x="species", y="sepal_width", data=iris, ax=axes[0, 1])  
sns.boxplot(x="species", y="petal_length", data=iris, ax=axes[1, 0])  
sns.boxplot(x="species", y="petal_width", data=iris, ax=axes[1, 1])
```

- The pairplot gives a quick global view of separability and overlap.
- The boxplots let students reason feature by feature instead of only visually scanning clusters.

Notebook step 4: correlation and class distribution

```
plt.figure(figsize=(8, 6))
sns.heatmap(iris.drop("species", axis=1).corr(),
            annot=True, cmap="coolwarm")
plt.title("Correlation Matrix of Iris Features")
plt.show()

print(iris["species"].value_counts())
sns.countplot(x="species", data=iris, hue="species", palette="Set2")
```

- This matches the notebook's explicit check of both feature correlation and label balance.
- It creates a natural comparison with future datasets where class balance is not this clean.

Notebook step 5: grouped summaries and first hypotheses

```
iris.groupby("species").describe().T

iris.groupby("species").mean().plot(kind="bar", figsize=(10, 6))
plt.title("Mean Feature Values by Species")
plt.ylabel("Mean Value")
plt.show()
```

- Grouped descriptive statistics connect plotting with interpretation.
- In the notebook, this is where students can argue that petal features separate species more clearly than sepal features.

What a good EDA output should contain

- Descriptive summaries for numerical and categorical variables.
- Missing-value patterns and candidate treatments.
- Class balance or target behavior overview.
- Correlation or dependency checks, including leakage suspicion.
- A short list of modeling risks and feature ideas.

Common beginner mistakes

- Starting model training before understanding the dataset.
- Confusing a pretty chart with an informative chart.
- Treating missing values as an inconvenience rather than a signal.
- Ignoring the business meaning of columns because the code still runs.
- Forgetting to document why a transformation was applied.

How the exercise notebook extends the workflow

- 'Class2_EDA_Exercise.ipynb' asks students to repeat the same sequence on Wine, Breast Cancer, and Digits.
- This turns one notebook into a reusable method instead of a one-dataset example.
- The key idea is transfer: the workflow stays stable even when the data semantics change.

Exercise pattern for scikit-learn datasets

```
from sklearn.datasets import load_wine

wine = load_wine(as_frame=True)
df = wine.frame

df.head()
df.info()
df.isnull().sum()
df["target"].value_counts()
```

- This slide mirrors the exercise notebook and shows how the same EDA structure transfers to another source.
- It also introduces the difference between a built-in seaborn dataset and a packaged scikit-learn dataset.

Papers and materials

[pandas documentation](#); [Matplotlib documentation](#); [Jupyter install guide](#); [scikit-learn documentation](#).

From This Class to the Next

What stays with us

- EDA is not decoration; it is how we validate assumptions before modeling.
- Python, pandas, and plots make data quality, target balance, and suspicious patterns visible.
- Good exploration produces questions for preprocessing, feature design, and evaluation.

Project use

In the fraud and pasteurization cases, this is the stage where we inspect ranges, missingness, imbalance, time behavior, and candidate signals.

Next connection

Class 11 uses this same habit of inspection right before fitting anything, as we take first contact with the scikit-learn API for training and evaluating a model.

Course thread: We first learn to see the data clearly, then we decide how to encode, transform, or model it — the theory of that transformation is picked up again in the Data-Driven block.

Papers and materials

Pedregosa et al. (2011), scikit-learn: Machine Learning in Python; Tukey (1977), Exploratory Data Analysis; Sambasivan et al. (2021), Everyone wants to do the model work, not the data work; pandas overview.